

长期记忆对话 Agent 最终报告

队长：汤岑宸 学号：231880523

2026 年 6 月

1 任务目标

本项目实现一个支持长期记忆的对话 Agent。系统需要从多会话历史中抽取可管理的记忆单元，在回答新问题时检索相关记忆，并通过去重、更新或遗忘等机制维护记忆状态。与直接拼接原始对话不同，本项目关注的是将原始对话日志加工为可追踪、可检索、可更新的长期记忆。

评测采用基于 LoCoMo 的长期对话问答数据集，问题类型包括 single-hop、multi-hop、temporal 和 open-domain。主指标为 LLM-as-Judge 得分，同时报告平均每题 LLM 调用次数和平均端到端响应时间。

2 相关工作与设计动机

本系统主要参考三类长期记忆 Agent 工作。

Generative Agents 提出了基于 relevance、recency 和 importance 的三因子记忆检索机制。该思想适合长期对话任务：仅依靠语义相似度可能召回较旧或不重要的信息，而时间和重要性可以帮助系统优先选择更有用的记忆。因此，本项目在 Memory Retriever 中实现了三因子加权排序。

MemoryBank 强调记忆随时间变化的动力学，并使用遗忘曲线建模记忆强度。考虑到本项目首先需要一个稳定可跑的系统 baseline，当前版本未实现完整 Ebbinghaus 遗忘曲线，而是使用轻量 recency decay 作为时间衰减近似；后续可在此基础上扩展为显式遗忘机制。

Mem0 将记忆系统拆分为提取、冲突检测和更新等工程流程。该流程对本项目很有启发：系统不应把原始对话直接当作最终记忆，而应先抽取事实性记忆，再进行去重和更新。因此，本项目在 Memory Writer 中使用 LLM prompt 抽取结构化记忆，并在 Memory Updater 中实现轻量去重和同槽位更新。

3 系统架构

最终系统由五个模块组成：Memory Writer、Memory Store、Memory Retriever、Memory Updater 和 Agent Controller。整体流程为：

1. ingest 阶段读取多会话历史。
2. Memory Writer 从每个 session 中抽取结构化记忆。
3. Memory Updater 对候选记忆进行去重与简单更新。

4. Memory Store 保存记忆并构建 embedding 索引。
5. **answer** 阶段根据问题检索相关记忆，组成 prompt 后调用 LLM 生成答案。

3.1 Memory Writer

Memory Writer 负责把原始对话转化为长期记忆单元。当前实现使用 LLM prompt 对每个 session 进行抽取，要求模型返回 JSON 数组。每条记忆包含以下字段：

- **text**: 独立、简短的事实性记忆句子；
- **subject**: 记忆涉及的人或实体；
- **attribute**: 记忆类型，如 preference、location、job、event、relationship、plan、update 等；
- **importance**: 1 到 5 的重要性分数；
- **source** 和 **timestamp**: 用于追踪记忆来源和时间。

该模块参考 Mem0 的 memory extraction 思路，但实现保持简单：每个 session 分块抽取，解析失败时使用关键词规则兜底，避免整条流水线因单次格式错误中断。

3.2 Memory Store

Memory Store 负责保存记忆及其索引。当前实现使用内存列表保存 **MemoryItem**，并使用本地 BAAI/bge-m3 生成向量表示。每条记忆保存文本、来源、时间戳、重要性、元数据、插入顺序和访问次数。检索前，Store 会批量编码所有记忆并构建 embedding 矩阵。

当前版本没有接入 FAISS 或 Chroma，而是直接使用矩阵乘法计算 query 与记忆向量之间的余弦相似度。这一选择实现简单、便于调试，也足够支撑当前 200 题规模的实验。后续若扩展到更大数据集，可以将该模块替换为 FAISS 索引，而不改变上层接口。

3.3 Memory Retriever

Memory Retriever 参考 Generative Agents，使用三因子加权排序：

$$\text{Score}(m, q) = 0.70 \cdot \text{Relevance}(m, q) + 0.15 \cdot \text{Recency}(m) + 0.15 \cdot \text{Importance}(m)$$

其中，Relevance 由 bge-m3 embedding 相似度计算；Recency 根据记忆时间戳做指数衰减，越新的记忆分数越高；Importance 来自 Writer 抽取阶段的 1-5 分重要性评分并归一化。当前默认返回 top-8 条记忆。

该设计比 Vanilla RAG 更进一步：Vanilla RAG 直接检索原始对话切片，而本系统检索的是经过抽取和更新的记忆单元，并且在相似度之外考虑时间和重要性。

3.4 Memory Updater

Memory Updater 实现轻量的记忆管理机制。当前版本包含两步：

1. 文本级去重：对规范化后的相同记忆只保留一条，并保留较高 importance。

2. 同槽位更新：若两条记忆具有相同的 `subject + attribute`，则保留时间更新的记忆。

该机制对应 Mem0 中的 `conflict/update` 流程，但没有引入额外 LLM 判别器，因此成本较低。它适合处理偏好、地点、职业、计划等可能随时间变化的信息。例如，当同一用户的 `location` 更新时，较新的 `location` 记忆会替代旧记忆。

3.5 Agent Controller 与日志

Agent Controller 串联完整流程。在 `ingest` 阶段，Controller 调用 Writer、Updater 和 Store；在 `answer` 阶段，调用 Retriever 得到相关记忆，并构造回答 prompt。

系统会记录每题的 `trace`，包括：

- 检索到的记忆内容；
- 每条记忆的总分、`relevance`、`recency` 和 `importance` 分项；
- 完整 prompt；
- LLM 调用统计。

这些日志用于后续 `bad case` 分析，可以将错误归因到写入失败、检索失败或生成失败。

4 实验设置

4.1 模型与数据

生成阶段使用本地 `Qwen2.5-3B-Instruct-AWQ`，通过 vLLM 提供 OpenAI 兼容接口。Vanilla RAG 和本系统的 `embedding` 模型均使用本地 `BAAI/bge-m3`。Judge 阶段使用 DeepSeek API 的 `deepseek-v4-flash`。

当前评测集包含 10 段对话、200 个问题，四类问题各 50 个。所有方法使用同一生成模型，保证差异主要来自记忆使用方式。

4.2 对比方法

表 1: 对比方法

方法	记忆使用方式	说明
No-memory	不使用历史对话	仅根据当前问题回答
Full-context	截断历史上下文	将原始对话历史拼入 prompt
Vanilla RAG	原始对话切片检索	使用 <code>bge-m3</code> 检索 top-k 原始片段
Ours	结构化记忆检索	LLM 写入记忆，三因子检索，轻量更新

5 Baseline 结果

目前已完成三组 baseline 的正式评测。表 2 给出总体结果。

表 2: Baseline 总体结果

方法	Judge Score	F1	EM	平均 LLM 调用	平均端到端时间
No-memory	0.0025	0.0000	0.000	1.0	1.043s
Full-context	0.0950	0.1013	0.060	1.0	1.352s
Vanilla RAG	0.0850	0.0940	0.040	1.0	1.317s
Ours	待补充	待补充	待补充	待补充	待补充

结果显示, No-memory 基本无法处理长期记忆问题。Full-context 当前略高于 Vanilla RAG, 说明直接检索原始对话切片并不必然优于截断上下文; 其原因可能是 top-k 原始片段缺少上下文, 或 multi-hop、temporal 问题需要多个证据共同推理。该结果也说明, 后续系统不能只依赖简单 RAG, 而需要更好的记忆写入和检索排序。

表 3: Baseline 按问题类型拆分的 Judge Score

方法	multi-hop	open-domain	single-hop	temporal
No-memory	0.010	0.000	0.000	0.000
Full-context	0.060	0.220	0.080	0.020
Vanilla RAG	0.040	0.160	0.120	0.020
Ours	待补充	待补充	待补充	待补充

分类结果表明, open-domain 和 single-hop 相对更容易, 而 multi-hop 和 temporal 仍然较弱。这与长期记忆任务本身的难点一致: multi-hop 需要跨多个会话片段聚合证据, temporal 需要判断事件时间、更新顺序和最新状态。

6 后续实验与消融计划

本系统已经具备完整模块, 后续可以围绕以下方向做消融:

1. **检索策略消融**: 比较只使用 relevance 的检索与 relevance + recency + importance 三因子检索。
2. **写入方式消融**: 比较直接原始切片检索、规则式记忆写入和 LLM 结构化记忆写入。
3. **更新机制消融**: 比较只追加记忆与启用 subject/attribute 同槽位更新。
4. **top-k 消融**: 比较 top-5、top-8、top-12 对不同题型的影响。

最终报告将补充 Ours 的整体准确率、分类型准确率、成本指标和端到端时延, 并从失败样例中挑选 3-5 条进行 bad case 分析。bad case 将按写入失败、检索失败和生成失败三类归因。

7 总结

本项目已经完成三组 baseline 和一个结构化长期记忆 Agent 的系统实现。自定义系统参考了 Generative Agents 的三因子检索、MemoryBank 的时间衰减思想和 Mem0 的记忆提取与更新流程,

同时保持实现轻量，便于后续做消融和错误分析。当前 baseline 结果显示，简单 Vanilla RAG 尚未稳定超过 Full-context，因此最终系统的关键在于提升记忆写入质量、召回完整性和时间更新能力。